Computer grafica - HTML

Interfacce grafiche

Lezione del 29/9/2020

Argomenti:
Impaginazione di contenuti: creazione di pagine statiche utilizzando HTML/CSS/SVG
Front-end interattivi: costruzione di applicazioni lato clien basate sul linguaggio JS ed il framework React
Applicazione Web: Realizzazione di back-end in NodeJS/MongoDB

User experience:

Prodotti e interazione

- Utilizziamo una vasta gamma di prodotti per supportare il nostro agire quotidiano:
  - sono il risultato di un agire specifico di persone che li hanno immaginati, progettati, realizzati, messi in campo.
  La messa in campo è molto importante, perché perdura nel tempo, finchè qualcuno lo usa.
  -I prodotti sono pensati per un fine preciso, ma molto spesso vengono usati per motivi diversi da quelli per cui vengono progettati
- Il loro successo è legato ad un insieme di fattori che è fortemente legato al modo con cui le persone entrano in contatto con esso
  -funzionalità, usabilità, versatilità, gradevolezza, robustezza, costo, margine di guadagno…
- Sono spesso il risultato di un processo molto complicato di affinamento ed evoluzione

Applicazioni come prodotti

- Il software, pur essendo un bene immateriale, è soggetto a molti meccanismi che regolano gli altri prodotti
  - La natura astratta delle azioni che supporta, rende spesso l'interazione uno degli aspetti fondamentali
- Occorre acquistare la capacità di progettare tale interazione in modo che risulti al tempo stesso:
  - Efficace
  - Efficiente
  - Facile da imparare
  - Piacevole (design)

Progettare dell'interazione

- Tradizionalmente, l'interazione tra persone e computer si è orientata al paradigma WIMP (windows, icons, menus, pointer)
- Con l'avvento dei dispositivi mobili, la diffusione degli schermi capacitivi e la crescita di velocità di elaborazione/comunicazione, tale paradigma è progressivamente cambiato
  - Comprendendo forme di interazione più articolate e, in parte, inclusive
- Le applicazioni software supportano le persone nei propri processi quotidiani
  - Dovrebbero rendere loro la vita semplice, fornendo un aiuto nei compiti cognitivi complessi, quali cercare, decidere, calcolare, prevedere,…
- Questo presuppone la capacità del progettista di
  - Calarsi nei panni degli utenti dei sistemi che si vogliono realizzare e guardare la realtà dal LORO punto di vista

- Prendere coscienza dei meccanismi di funzionamento/limiti intrinseci che contraddistinguono l'essere umano per evitare di sottoporre le persone a stress inutili

Immedesimarsi nell'utente

- I sistemi hanno molti utenti
  - Spesso svolgono ruoli differenti (amministratori, utenti finali, coordinatori, acqurenti, venditori, studenti, docenti, …) e hanno responsabilità/limiti legati a tale ruolo
- Ogni utente è un universo a sé
  - L'esperienza pregressa varia, la capacità di intuire molto di più
  - Ciascuno di noi ha le proprie preferenze/modelli mentali
- Il progettista NON è l'utente tipico!

Lezione del 30/9/2020

Interfacce grafiche

- L'interfaccia grafica costituisce, nel software, lo strato "visibile" dell'intero prodotto
  - La valutazione della sua qualità costituisce spesso un fattore di (pre-)giudizio che condiziona l'apprezzamento del prodotto nel suo complesso
  - Occorre sapere declinare accanto alle qualità più ingegneristiche (funzionalità, affidabilità, robustezza, scalabilità, …) qualità estetiche e di piacevolezza d'uso che le sottolineino

Comprensibilità dell'interfaccia

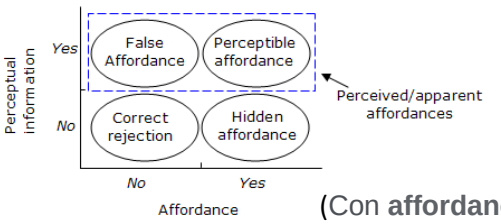
- Come nel caso di oggetti reali, ciò che viene esposto in un'interfaccia dovrebbe presentare delle "affordance"
  - Offrire un suggerimento naturale su come sia possibile interagire con esso
  - L'utente deve **rendersi conto che si può interagire con l'interfaccia**, che qualcosa può essere cliccato, scritto, inviato…

Coerenza dell'interfaccia

- Serve coerenza tra i suggerimenti visuali e le funzionalità reali

Altrimenti si genera frustrazione, attrito, …

La scelta dello stile visuale deve aiutare gli utenti nel processo di interpretazione del sistema



(Con **affordance** (invito all'uso) si definisce la qualità fisica di un oggetto che suggerisce a un essere umano le azioni appropriate per manipolarlo

Le diverse forme di abilità

- Nel portare avanti i processi quotidiani, ciascuno di noi fa leva sull'insieme di abilità che maggiormente gli appartengono

Linguistiche, logico-matematiche, musicali, cinestetiche, spaziali, interpersonali, introspettive, …

- L'interfaccia dovrebbe fornire suggerimenti adeguati per le diverse forme di abilità

Permettendo a ciascuno di orientarsi nel panorama dell'applicazione utilizzando il paradigma per sé più efficace (per esempio sul computer ci sono diversi modi per fare copia e incolla)

Altri aspetti legati all'interazione utente

- I meccanismi fisiologici legati alla percezione condizionano largamente ciò che raggiunge la nostra coscienza

Legge della gestalt (il nostro cervello procede dall'insieme al particolare, ma fermandosi appena può) quando noi guardiamo i tasti della tastiera, non vediamo i tasti, ma un agglomerato di cose, l'occhio si accontenta di sapere che lì ce una tastiera, non delle lettere in sé. In un gruppo troviamo piu facilmente le pecore nere che quelle bianche

Legge dell'attenzione selettiva: ci dice che noi tendiamo a guardare certe cose piuttosto che altre

- La capacità di reazione motoria di una persona è legata logaritmicamente all'intensità dello stimolo

Legge di Fitts: se si scrive premi qui in un angolo dello schermo molto piccolo e il puntatore dall'altra parte allora, costerà molta fatica. Se il pulsante fosse molto grande allora la reazione motoria sarebbe molto grande

- È possibile minimizzare lo sforzo cognitivo riorganizzando le interfacce così da rendere meno evidenti quei compiti che avvengono più raramente

Principio della progressive disclosure

Interfacce come risultato di un processo

- Tutti gli aspetti citati evidenziano come la creazione di un'interfaccia utente sia il risultato di un processo complesso

Che richiede una molteplicità di competenze, provenienti da discipline differenti

E che privilegia la figura degli utenti finali rispetto allo sforzo dei progettisti/sviluppatori

- In mancanza di tali fattori, l'interfaccia potrà anche funzionare

Ma genererà una percezione negativa del prodotto che vanificheranno gli sforzi di sviluppo

Interfacce web

Introduzione alle interfacce web

- Pensato inizialmente come strumento per la visualizzazione di documenti tra loro collegati, il paradigma ipertestuale del web è diventato strumento privilegiato per la creazione di interfacce utente grafiche

Offrendo strumenti avanzati per la visualizzazione e la manipolazione di contenuti informativi

Abilitando l'accesso remoto e concorrente a funzionalità ospitate all'interno della rete (cloud)

Permettendone la fruizione su dispositivi estremamente eterogenei (pc, smartphone, smart tv, tablet, …)

Principio generale

- Un browser permette di impaginare, visualizzare ed interagire con contenuti prelevati da un server web

E di interagire successivamente con lo stesso server per richiedere ed inviare dati inseriti/richiesti dall'utente

- Questo consente di rendere disponibile l'interfaccia

Senza richiedere installazione di software

In modo indipendente dal sistema operativo (ma dipendente dal browser)

- L'applicazione software di cui si vuole fornire l'interfaccia è ospitata lato server, pubblicata ad un indirizzo noto

URL – Uniform Resource Locator

- Collegandosi alla URL, il browser scarica una serie di moduli che contengono

La struttura generale dell'interfaccia (HTML)

Le regole di stile per presentarla (CSS)

Codice che reagisce alle azioni dell'utente e permette di trasformare le interazioni in richieste verso il server (JavaScript)

Dati relativi all'applicazione che vengono visualizzati all'interno dell'applicazione (JSON, XML, testo, immagini, …)

- Le interazioni tra client e server avvengono tramite il protocollo HTTP (HTTPS)

Comandi base HTML

```
<head>
        <meta charset="UTF-8"> permette di usare lettere come è
        <title> FINESTRA </title>  cambia il nome della scheda
</head>

<body>
```

<h1> TITOLO </h1> *esistono h1>h2>…>h6*

 … *paragrafo, di possono inserire attributi come:*

 se invece si vogliono inserire all’interno di un altro tag si fa

 testo

<img src= “URL” (URL se assoluto http://...) alt= “nome sostitutivo” (nome che sarà letto per i non vedenti)

LISTE:

 liste non numerate

 liste numerate

 elemento 1

scritte come > < ‘ “ & devono essere scritte in modo diverso

< (minore di)	<
> (maggiore di)	>
& (e commerciale)	&
" (virgoletta)	"

<!—commento-->

TABELLE

<table> *posso aggiungere un attributo come “border=”1””*

<tr>

<td> Alfa </td>

<td> Beta </td>

</tr>

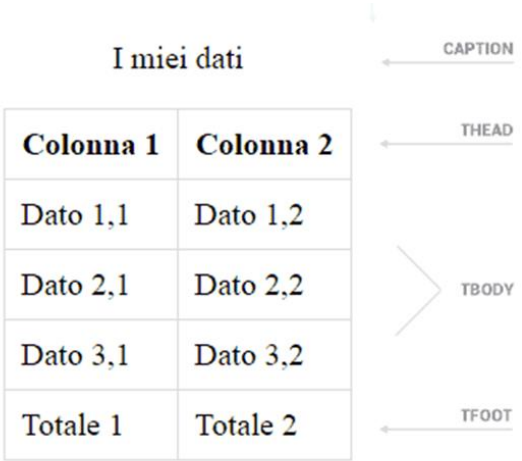
<tr>

<td> Gamma </td>

<td> Delta </td>

</tr>

Alfa	Beta
Gamma	Delta



Non è chiaro come funzioni <colgroup>

COLLEGAMENTI:

 Nome del link *(come su wikipedia)*

FORM

<form>

 Nome: <input type=”text” name= “nome”> *quando processeremo i dati serviranno a elaborare il codice*

* * <button> invia </button>

come attributi di input posso usare anche altre cose come:
type= “checkbox”
type= “radio” *(scelta multipla, devono avere lo stesso name= “x”)*
type= “submit” *(invia i dati)*
type= “password” *(son si vede quello che digita, ma solo ******)*

volendo si può far comparire un valore predefinito nel form
<input type= “text” name= “nome” value= “XXX”> *è una risposta precompilata*
oppure
<input type= “text” name= “nome” placeholder= “XXX”> *è la scritta grigina*

<textarea> … </textarea> *permette di scrivere un paragrafo*

<select>

<option> alfa </option>

<option> beta </option>

<select> *permette di creare un menù a tenda*

Il sistema non sa che nome: è riferito al campo input, per farlo si fa:

* *

<label for= “id1”> nome: </lable>

<input id= “id1” …>

Lezione del 06/10/2020

CSS – Cascading Style Sheet

- L'aspetto grafico di ciascun elemento è controllato dal suo stile
    - Insieme di coppie attributo:valore che definiscono dimensioni, posizione, spaziatura, colore, …
    - Sono presenti oltre 350 attributi differenti
- Ogni browser dispone di proprie impostazioni base legate a ciascun elemento
    - Queste possono essere modificate impostando una o più specifiche voci dello stile
    - Inserendo un insieme di regole che vanno a sostituire/integrare le impostazioni del browser

Regole di stile

- Lo stile è definito attraverso regole
    - Associano ad uno o più selettori un insieme di coppie attributo:valore, terminate da punto e virgola e racchiuse tra parentesi graffe
    - Il selettore indica su quali elementi debbano essere impostati i valori della regola
    - Esistono molti modi per selezionare gli elementi

Selettori

- Selettori di tipo
    - Indicano direttamente il nome di un tag, senza <>
    - Tutti gli elementi corrispondenti verranno influenzati dalla regola associata
    - Es: p { border: 0px; }
- Selettori di discendenza
    - Indicano due o più nomi di tag, separati da spazio
    - Si leggono da destra verso sinistra
    - Influenzano gli elementi del tipo più a destra, solo se sono discendenti (contenuti) negli elementi via via più a sinistra
    - Es: ul li { color: red; }
- Selettore universale
    - Si applica a tutti gli elementi indistintamente
    - Es: * { color: #000000; }

- Selettori di classe
    - Ogni elemento html può contenere l’attributo "class"
    - Esso contiene uno o più identificatori (separati da spazio) cui è possibile associare regole di stile
    - La sintassi di questo tipo di selettore è costituita dal nome della classe preceduto da "."
    - Es: .black { color: #000000; }
    - Lorem ipsum
- Selettori di elemento e classe
    - Restringono la validità della regola ai soli elementi il cui tipo e classe corrispondono al selettore stesso
    - Es: p.black { color: #000000; }
- Selettori di ID
    - Ogni elemento html può contenere l’attributo "id"
    - Esso deve contenere un identificatore univoco all’interno del documento
    - Il selettore corrispondente è preceduto da "#"
    - Es: #titolo { border: 0px; }
    - <h1 id="titolo">Lorem ipsum</h1>
- Selettori di elemento e classe
    - Restringono la validità della regola ai soli elementi il cui tipo e classe corrispondono al selettore stesso
    - Es: p.black { color: #000000; }
- Selettori di ID
    - Ogni elemento html può contenere l’attributo "id"
    - Esso deve contenere un identificatore univoco all’interno del documento
    - Il selettore corrispondente è preceduto da "#"
    - Es: #titolo { border: 0px; }
    - <h1 id="titolo">Lorem ipsum</h1>
- Selettore di discendenza diretta
    - Selettore composto che si applica a tutti gli elementi identificati dal tipo più a destra, se direttamente contenuti dal tipo più a sinistra
    - Es: body > p { color: #000000; }
- Selettore di attributo

    - Si applica a tutti gli elementi del tipo specificato che contengono l'attributo indicato tra "[]" con il relativo valore
    - Es: input[type="text"] {color: #000000;}
    - È possibile indicare che l'attributo deve contenere una data stringa (attr ~= "val") o che deve iniziare con quella stringa (attr |= "val")

- Raggruppamento di selettori
    - Si applica a tutti gli elementi che corrispondono ad uno dei selettori indicati
    - Es: h1, h2, h3 {color: #000000;}
    - #footer, #header {width: 400px;}

- Raggruppamento di selettori
    - Si applica a tutti gli elementi che corrispondono ad uno dei selettori indicati
    - Es: h1, h2, h3 {color: #000000;}
    - #footer, #header {width: 400px;}

- Pseudo-selettori
    - Indicano il nome di un tag seguito da una parola-chiave che inizia con ':'
    - Si applicano agli elementi indicati dal tag solo quando si trovano nello stato indicato dalla parola chiave
    - Es: li:hover {color: red;}

  Possibili parole chiave
    - :hover - Il mouse si trova nell'area dell'elemento
    - :focus - l'elemento possiede il focus della tastiera
    - :first-child - l'elemento è il primo figlio del suo elemento contenitore

Fogli di stile

- I fogli di stile vengono distinti in due tipologie
    - Interni quando il loro codice è compreso in quello del documento html
        - Possono essere in linea o incorporati
    - Esterni quando sono definiti in file separati dal documento con l'estensione .css
        - Si definiscono collegati
- In linea
    - Quando si trovano all'interno dei singoli tag
      <h1 style="color: #98c7d4;">Title</h1>

- Incorporati
    - Se sono inseriti direttamente nell'<head> del documento tramite il tag <style>

```
<head>
        <style>
            h1 { color: #98c7d4; }
        </style>
</head>
```

- Collegati
    - Utilizzando l'elemento HTML <link>

```
<link rel="stylesheet" href="styles.css"/>
h1 { color: #98c7d4; }
p { font-size: 12px; }
.highlight {
  background-color: #ffff00;
  color: #c0c000;
}
```

Ordine a cascata

- La priorità dello stile è determinata dalla sua posizione nel sito.
- In ordine, dalla meno alla più importante:
    - Default del browser
    - Link esterno
    - Head
    - Attributo inline
    - Utilizzo di !important
- La priorità dipende anche dalla posizione nel documento.
    - L'ultima definizione sovrascrive le precedenti

```
.intro {
color: #444245;
}
 .intro {
color: #dddadd;
 }
```

Lezione del 07/10/20

Impaginazione

- Il motore CSS provvede a raggruppare tutto il contenuto presente all'interno di un documento e a disporlo opportunamente all'interno della superficie di visualizzazione
    - Il risultati finale è condizionato da un certo numero di proprietà di stile dei singoli elementi…
    - …oltreché dal contenuto del documento stesso
- L'algoritmo di impaginazione assegna a ciascun elemento html uno o più rettangoli sulla superficie di visualizzazione
    - Determinandone origine (x,y), dimensioni (larghezza e altezza) e livello di stratificazione (z)

Box Model

- I singoli rettangoli assegnati agli elementi possono essere estesi impostando opportune proprietà
    - Padding, border, margin



Padding

- Estensione del rettangolo occupato dal contenuto di un elemento volta a creare una spaziatura attorno ad esso
    - Non può avere valori negativi
    - Normalmente, l'area di padding è riempita con lo stesso sfondo impostato sull'elemento
    - Agendo sulle proprietà background-* è possibile modificare tale comportamento
- Il padding può avere valori diversi per ciascun lato
    - Per impostarli è possibile definire ciascuno di essi singolarmente (padding-top, padding-right, padding-bottom, padding-left) oppure tramite la proprietà *shorthand* padding e indicandogli i valori nell'ordine top-right-bottom-left (senso orario)
    - Nel caso in cui si voglia impostare un valore uguale per top e bottom e un valore uguale per right e left, è sufficiente indicare soltanto due valori
        - ```
          .element {
          padding-top: 15px;
          padding-right: 15px;
          padding-bottom: 10px;
          padding-left: 10px;
          }
          ```
        - ```
          .element {
           padding: 15px 10px 15px 10px;
          }
          ```

```
.element {
 padding: 15px 10px;
}
```

- Il padding può essere indicato in maniera assoluta, ovvero con i “pixel” indipendentemente dal font, ma è meglio farlo in maniera relativa usando “em”

Border

- Anche il border può avere valori diversi per ciascun lato
    - border-width specifica lo spessore del bordo, può essere indicato con un valore numerico con unità di misura oppure con le parole chiave (medium, thin, thick)
    - border-style specifica lo style del border (none, hidden, dotted, dashed, solid, double)
    - border-color specifica il colore del bordo
    - initial imposta l’attributo al valore di default definito dal browser
    - inherit eredita il valore dall'elemento genitore
- Valgono le stesse regole del padding
- Anche il border può avere valori diversi per ciascun lato
    - border-width specifica lo spessore del bordo, può essere indicato con un valore numerico con unità di misura oppure con le parole chiave (medium, thin, thick)
    - border-style specifica lo style del border (none, hidden, dotted, dashed, solid, double)
    - border-color specifica il colore del bordo
    - initial imposta l’attributo al valore di default definito dal browser
    - inherit eredita il valore dall'elemento genitore
- Valgono le stesse regole del padding

Margin

- La proprietà margin è utilizzata per generare spazio attorno agli elementi, impostandone la dimensione
- È una proprietà *shorthand* (margin-top, margin-right, margin-bottom, margin-left)
    - auto il valore del margin viene calcolato dal browser
    - length definisce un margine utilizzando valori numerici con unità di misura (5px, 3mm…)
    - % definisce un margine con un valore in percentuale relativo all’elemento genitore
    - inherit eredita il valore dall'elemento genitore
- I margini non si sommano, bensì si compensano

Box Model

- Per calcolare la width totale del box è necessario sommare contenuto, padding e border

- ```
  .element {
    border: 5px solid #fff;
    padding-left: 10px;
    padding-right: 5px;
    width: 100px;
  }
  ```

```
 content width 100px +
 padding width  15px +
border width    10px =
-----------------------
      box width   125px
```

Overflow

- La proprietà overflow definisce cosa accade nel caso in cui il contenuto di un elemento fuoriesca dal suo box.
- Può avere quattro valori:
  - visible valore default, consente al contenuto di estendersi oltre i confini del container
  - auto aggiunge scrollbar se necessario
  - hidden nasconde il contenuto che va oltre il contenitore
  - scroll aggiunge scrollbar anche se non sono necessarie

Impaginazione

Abbiamo usato l'elemento "div" che nell'html è di bassissima semantica e indica semplicemente un pezzo di contenuto che deve essere rappresentato come blocco, ovvero ha come elemento base block; Il suo duale è l'elemento "span", ovvero un elemento che ha associato come proprieta il css display

<div
display: inline
…>
è uguale a usare <span>

- Le principali proprietà che influiscono sull'impaginazione sono
  - display block, inline, inline-block, none …
  - position static, relative, absolute, fixed
  - float left, right, none

Display

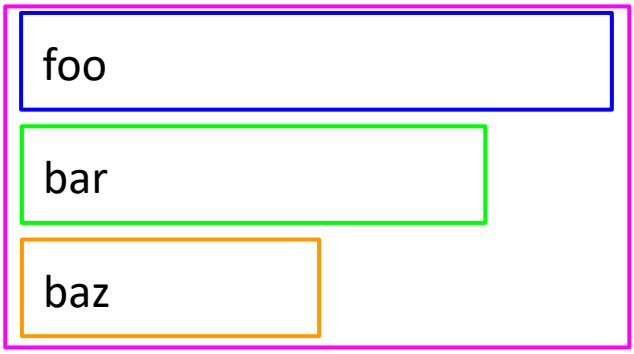
- Controlla quale algoritmo debba essere usato per disporre il contenuto presente all'interno di un dato elemento
  - block il contenuto disposto all'interno di un rettangolo che occupa, orizzontalmente, tutto lo spazio disponibile nel proprio elemento contenitore
  - inline il contenuto è suddiviso in una o più linee, occupando, su entrambe le direzioni, lo spazio strettamente necessario
  - inline-block all'elemento viene assegnato un solo rettangolo, come nel caso dei block, ma tale rettangolo può essere affiancato orizzontalmente da altro contenuto
  - none il contenuto non viene visualizzato
```

Posizionamento di blocchi

- Quando un elemento di tipo block contiene altri elementi dello stesso tipo, questi vengono disposti verticalmente, a partire dall'angolo in alto a sinistra del contenitore
    - I blocchi contenuti avranno il bordo sinistro coincidente con il blocco sinistro del proprio contenitore
    - L'attributo width ne controlla la larghezza

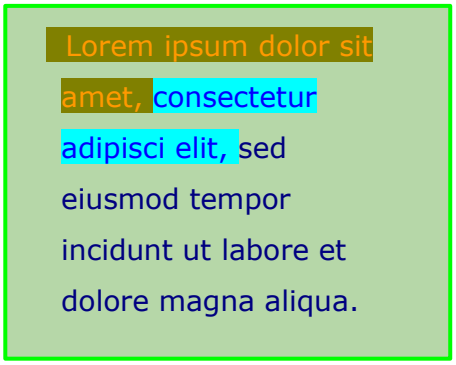
```
.bar {  width: 75%; }
.baz {  width: 50%;}

<div class="container violet">
  <div class="blue">foo</div>
  <div class="bar green">bar</div>
  <div class="baz orange">baz</div>
</div>
```



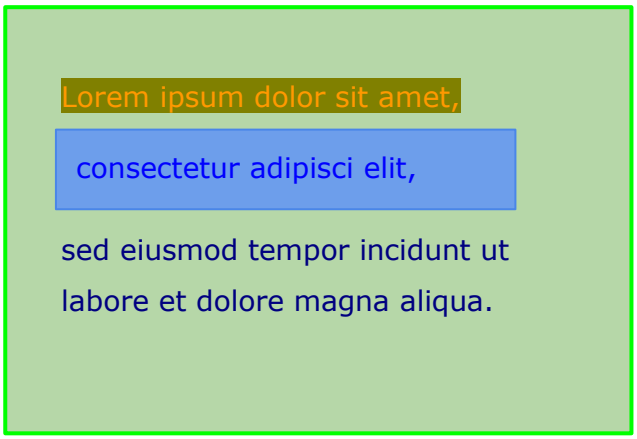
- Quando un elemento di tipo block contiene solo elementi di tipo inline o testo semplice, questi vengono disposti uno a fianco all'altro
    - Se c'è spazio a sufficienza
    - Altrimenti, vengono divisi in linee e ogni linea viene posizionata sotto le precedenti

```
<div class="container green">
  <span class="orange">Lorem ipsum dolor
      sit amet,</span>
   <span class="blue"> consectetur
      adipisci elit,</span> sed eiusmod
      tempor incidunt ut labore et dolore
      magna aliqua.
</div>
```



- Quando un elemento di tipo block contiene sia elementi di tipo inline (o testo semplice) che elementi di tipo block, i primi vengono inseriti in un blocco anonimo
    - Che viene inserito nel flusso degli altri blocchi presenti all'interno del contenitore

```
<div class="container green">
  <span class="orange">Lorem ipsum dolor
      sit amet,</span>
   <div class="blue"> consectetur
      adipisci elit,</div> sed eiusmod
      tempor incidunt ut labore et dolore
      magna aliqua.
</div>
```



Position

- Attributo che indica dove verrà collocato un dato rettangolo all'interno della superficie di visualizzazione
    - static valore di default, indica che l'elemento segue la regola precedentemente espressa
    - relative indica il rettangolo verrà collocato in una posizione sfalsata rispetto a quella di default
    - absolute indica che il rettangolo verrà posizionato in modo assoluto rispetto al rettangolo del suo primo elemento contenitore la cui posizione non sia static
    - fixed il rettangolo assume una posizione fissa rispetto alla superficie di visualizzazione

Posizionamento relativo

- Gli attributi top, left, right e bottom permettono di traslare i rispettivi lati del rettangolo associato all'elemento della quantità indicata
    - Possono assumere sia valori positivi che negativi
- I loro valori possono essere espressi
    - Con dimensioni assolute (pixel, cm, …)
    - Relativamente alla dimensione del font corrente (em)
    - Con dimensioni relative al contenitore (%), a condizione che questo abbia la corrispettiva dimensione fissata (tramite width e height)

Posizionamento Assoluto

- La posizione e le dimensioni di questo tipo di elementi sono definite tramite gli attributi top, right, bottom e left
    - Se assumono valori relativi (%) essi fanno riferimento alle dimensioni dell'elemento contenitore di riferimento (position!=static)

    - Spesso si etichetta l'immediato contenitore con position=relative, così da esprimere l'elemento di riferimento

Posizionamento sull'asse Z

- Impostando l'attributo position, è possibile alterare la posizione di un elemento e portarlo a sovrapporsi ad altri elementi
    - Si può controllare l'ordine di visualizzazione impostando l'attributo z-index
    - Maggiore è il valore numerico associato a z-index, più in alto apparirà l'elemento sulla superficie di visualizzazione

Float

- L'attributo float rimuove gli elementi dal normale flusso di impaginazione del documento e li sposta su uno dei lati del suo contenitore
- Può avere quattro valori:
    - none valore iniziale, l'elemento non è float
    - left sposta l'elemento alla sinistra del contenitore
    - right sposta l'elemento alla destra del contenitore
    - inherit l'elemento eredita la direzione float dell'elemento genitore
- Un elemento float è automaticamente display: block;
- Gli elementi float sono esclusi dal flusso normale di posizionamento
    - Non influenzano la posizione verticale degli altri blocchi
- Vengono accumulati sul lato sinistro (o destro) del proprio contenitore
    - Limitando la lunghezza delle linee di eventuali altri contenuti di tipo inline, presenti nello stesso contenitore
- L'altezza del blocco contenitore non è legata all'altezza dei blocchi float in esso contenuti

Floating

- Gli elementi float si accumulano fino al bordo dell'elemento genitore
    - Quando non hanno più spazio si spostano verso il bordo successivo disponibile
- Quando gli elementi hanno altezze diverse il comportamento degli elementi potrebbe essere diverso

Clearing

- Un elemento con la proprietà clear non si posizionerà in modo adiacente all'elemento float, ma si muoverà verso il basso
    - Ad occupare una linea il cui margine sia libero
- La proprietà clear serve a impedire che compaiano elementi al fianco di elementi float
- Si applica a elementi block e non è ereditata

- Può avere diversi valori:
    - none valore iniziale, permette agli elementi float di posizionarsi a destra e a sinistra
    - left impedisce il posizionamento a sinistra
    - right impedisce il posizionamento a destra
    - both impedisce il posizionamento su entrambi i lati
- Ci sono tre metodi
    - Metodo del div vuoto
        - Consiste nell'inserire un div con la proprietà clear:both;
    - Metodo Overflow
        - Questo metodo comporta l'inserimento della proprietà overflow sull'elemento contenitore
        - Se questa proprietà è impostata su auto o hidden, il genitore si espanderà per contenere i float, eliminandolo per gli elementi successivi
    - Metodo Clearfix
        - Questo metodo usa lo pseudo selettore :after sull'elemento contenitore per eseguire il clear
        - È necessario dare una classe all'elemento genitore e applicare questo css:

Lezione del 13/10/2020

Invece di fare il 3° blocco di slide dall'inizio è saltato a piè pari alla slide n°65

Flexbox

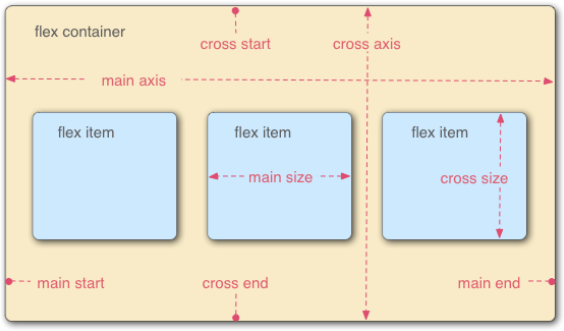
Il controllo di impaginazione in css è incasinato, per questo il css3 ha introdotto l'algoritmo di posizionamento Flexbox, viene abilitato impostando il display:flex

- Un elemento utilizza questo algoritmo di disposizione se la sua proprietà display è impostata al valore flex;
    - Questo elemento viene definito flex container
    - I flex items sono i suoi sottoelementi direttamente contenuti
    - Occupano lo spazio necessario in base al contenuto e tutto sulla stessa linea

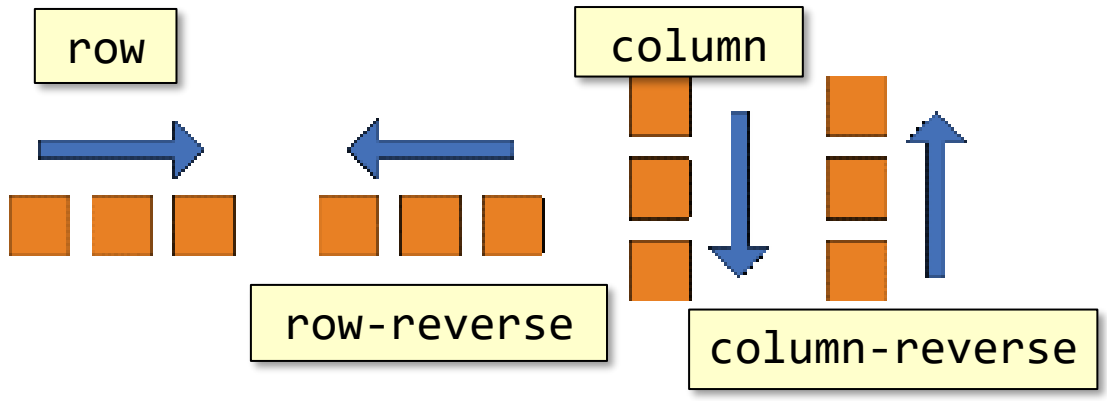
```
<div class="container">
 <div class="a">uno</div>
 <div class="b">due</div>
 <div class="c">tre</div>
</div>
```

```
.container {display:flex;}
.a { flex-grow:1; }
.b { flex-grow:2; }
.c { flex-grow:3; }
```





- main-axis l'asse principale del flex container sulla quale vengono disposti gli elementi
    - Non è necessariamente orizzontale, ma la sua direzione dipende dalla proprietà flex-direction
- Flex-grow permette di dividere in maniera proporzionale solo lo spazio residuo dell'intera riga
- Flex-basis dice quanto vale il contenuto del div, ovvero la dimensione che sarà da considerare per lo spazio residuo
- Flex-shrink dice cosa fare quando ce meno spazio(restringo la finestra), ovvero quante parole per riga



Flexbox – Flex container

- flex-wrap permette di disporre gli elementi su una o più linee parallele al main axis
    - nowrap, valore di default, tutti i flex items vengono posizionati su un'unica linea
    - wrap i flex items si posizionano su più linee, ordinate nella direzione del cross axis
    - wrap-reverse i flex items si posizionano su più linee nella direzione opposta a quella del cross axis

Lezione del 20/10/2020

CSS3

- CSS3 è il nuovo standard CSS che include
    - Nuovi selettori
    - Box Model
    - Nuovi effetti
    - Nuovi sistemi di posizionamento
- La proprietà border-radius permette di arrotondare gli angoli dei bordi

```
.box {
  border-radius: 15px;
}
.box {
  border-radius: 4px 15px 12px 10px;
  ( <top left> <top right> <bottom right> <bottom left>)

.box {
  border-radius: 50%;
} (con % fa riferimento alle dimensioni del box, quindi con il 50% sarà un ovale)
```

- La proprietà box-shadow applica un effetto ombra ad un elemento

```
.box {
  box-shadow: 1px 2px 2px #000;
}
```

box-shadow: <inset> <offset-x> <offset-y> <blur-radius> <spread-radius> <color>
*inset se non specificato viene creata una drop shadow
offset-x spostamento dell'ombra sull'asse x
offset-y spostamento dell'ombra sull'asse y
*blur-radius livello di sfocatura dell'ombra
*spread-radius livello di diffusione dell'ombra
color colore dell'ombra

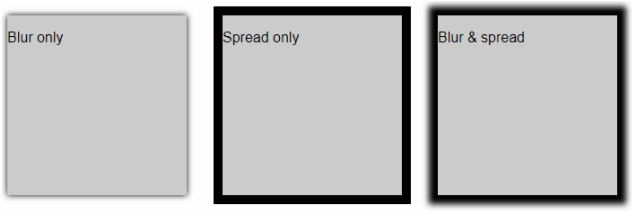
**parametri opzionali*

```
.box {
  box-shadow: 10px 20px 20px #000;
}
```



Nessun valore inset, quindi l'ombra sarà di tipo drop
10px offset-x
20px offset-y
20px blur-radius
Nessun valore spread-radius
#000 colore

- ```
  .box {
  box-shadow: 10px 20px 0 20px #000;
  }
  ```
  Se volessimo avere lo spread-radius al posto del blur-radius sarebbe sufficiente specificare blur-radius:0;



- È possibile definire più box-shadow separandole con una virgola
  ```
  .box {
    box-shadow:
      10px 20px 20px #000,
      inset 10px 10px 20px #7f4500;
  }
  ```



- La proprietà text-shadow è simile alla box-shadow ma applica un effetto ombra ad un testo
  ```
  h1 {
    text-shadow: 1px 2px 2px #000;
  }
  ```

- text-shadow:<offset-x> <offset-y> <blur-radius> <color>
  offset-x spostamento dell'ombra sull'asse x
  offset-y spostamento dell'ombra sull'asse y
  *blur-radius livello di sfocatura dell'ombra
  color colore dell'ombra

  *parametri opzionali

  N.B: questo appesantisce molto il sistema, quindi sarà usato difficilmente

- Nel CSS 2.1 la regola del box model comporta che se si impostano valori di width e height per un certo elemento, queste misure vengono applicate alla sola area del content, escludendo padding e border

- La proprietà del CSS3 box-sizing rende più flessibile il modello di base
  ```

- Valori dell'attributo box-sizing
    - content-box valore di default, imposta le dimensioni considerando il solo contenuto, escludendo border, margin e padding
    - padding-box include content e padding, ma esclude border e margin
    - border-box include content, padding e border, ma non il margin
- Il CSS3 consente di applicare sfondi multipli ad un elemento
    - Inizialmente è necessario elencarli separati da virgole
    - Successivamente bisogna specificare la background-position per ciascuno di essi
    - Infine specificare il background-repeat per ognuno

```
.element {
background-image: url(bg1.png), url(bg2.png);
background-position: top left, center right;
background-repeat: no-repeat, no-repeat;
 }
```

- È disponibile anche il comando veloce background

```
.element {
  background:
    url(corner1.png) top left no-repeat,
    url(corner2.png) bottom right no-repeat;
 }
```

- La proprietà color permette di lavorare sia con in RGBa sia in HSLa
- Il metodo RGBa (Red Green Blue alfa) lavora in sintesi additiva con i relativi colori primari rosso, verde e blu con in più il canale alfa che sta ad indicare l'opacità del colore
- Il metodo HSLa (Hue Saturation Lightness alfa) lavora con tonalità, saturazione e brillantezza, con in aggiunta il canale alfa

```
.element {
  color: rgba(0, 0, 0, 0.75);
}
.element {
  color: hsla(240, 100%, 50%, 0.75);
 }
```

- CSS3 permette di impostare l'opacità di un elemento attraverso la proprietà opacity
    - Questa proprietà influenza tutti gli elementi annidati sotto l'elemento alla quale si applica

```
.element {
  opacity: 0.45; //questo elemento è opaco al 45%
}
```

- È possibile creare delle sfumature tra due e più colori grazie alla proprietà gradient
- Ci sono due tipologie di gradienti
    - Lineari
    - Radiali

- Per creare una sfumatura lineare bisogna specificare un punto di partenza, un punto di fine e, quando necessario, dei colori intermedi

```
.element {
  background:
    linear-gradient(to bottom right, #ffffbb,
    rgba(255,255,255,0.0)) top left no-repeat;
}
```



linear-gradient
(<angle> to <side-or-corner>, <color-stop>s);

- *angle generalmente espresso in gradi
  *side-or-corner consiste in due parole chiave che indicano la direzione (left o right per l'asse orizzontale, top o bottom per l'asse verticale)
  color-stop consiste in una serie di almeno due colori

  **parametri opzionali*

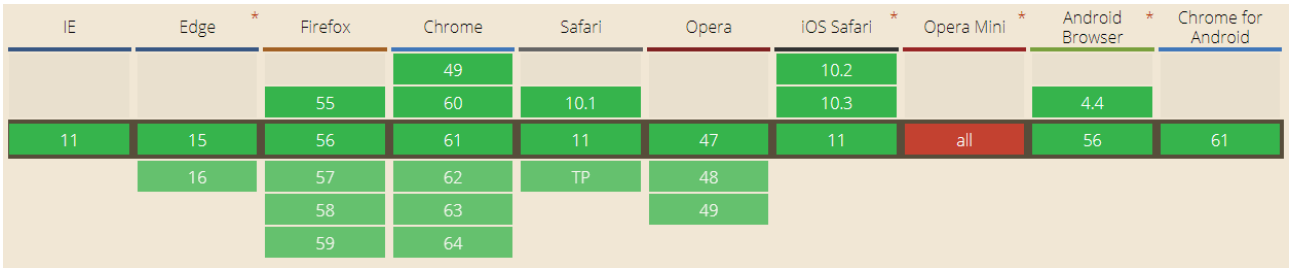
- Una sfumatura radiale, a differenza di una lineare, crea un gradiente che parte dal centro dell'elemento e si estende in maniera circolare o ellittica

```
.element {
  background:
    radial-gradient(#ffff00, rgba(255,255,255,0.0));
}
```


*shape indica la forma della sfumatura che può essere circolare (circle) o ellittica (ellipse, valore di default)
*size

- Utilizzando @font-face possiamo utilizzare font online per il nostro progetto
  - Questo permette di visualizzare il font su tutti i browser, anche quando non è installato sul pc dell'utente

```
@font-face {
 font-family: 'OpenSansRegular';
 src:
   url('OpenSansRegular-webfont.woff2')format('woff2'),
   url('OpenSansRegular-webfont.woff')format('woff');
}
```



- Per applicare il font ad un elemento è necessario specificarlo nello stesso modo in cui è stato specificato nella chiamata a font-face

```
h1 {
 font-family: 'OpenSansRegular';
}
```

Media query

- CSS permette di inserire blocchi di regole legate ad un particolare mezzo di presentazione
    - Introdotti dalla parola chiave @media seguita dal nome del mezzo (screen, print, …)

```
@media screen
body {
font-family: Verdana, Helvetica, sans-serif;
font-size: 0.8em;
}

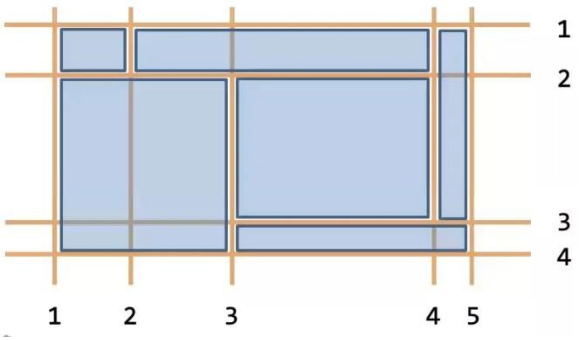
@media print
body {
font-family: “Times New Roman”;
font-size: 10pt;
}
```

- CSS3 permette di estendere questa sintassi, limitando l’applicabilità delle regole al verificarsi di ulteriori condizioni
    - Legate agli specifici parametri del dispositivo di visualizzazione
- Parametri controllabili
    - Larghezza e altezza della superfice di visualizzazione
    - Larghezza e altezza del display del dispositivo
    - Orientamento (portrait, landscape)
    - Rapporto tra le dimensioni (aspect ratio)
    - Risoluzione (densità dei pixel)

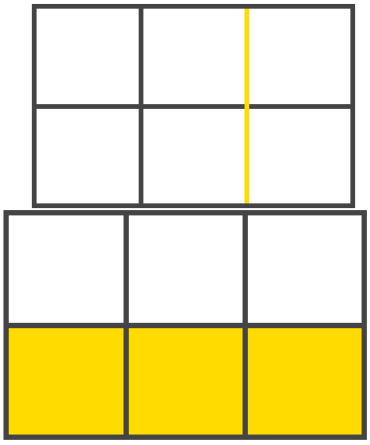
Responsive layout

```
@media screen and (max-width:480px) {
      body {
       font-size: 1rem;
       }
      }
      @media screen and (min-width:481px) and (max-width:650px) {
       body {
         font-size: 1.5rem;
       }
}
@media screen and (min-width:651px) {
body {
   font-size: 2rem;
 }
}
```

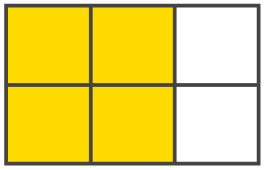
CSS GRID



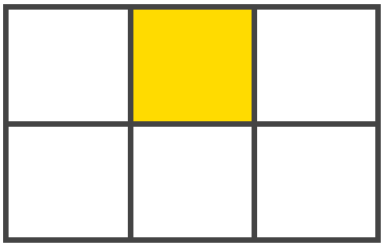
- Un elemento con la proprietà display: grid è detto Grid Container
    - Gli elementi direttamente contenuti al suo interno sono detti Grid Items
- Per utilizzare questo algoritmo, è necessario:
    - Definire un container come griglia con la proprietà display:grid;
    - Impostare le dimensioni di colonne e righe con grid-template-columns e grid-template-rows
    - Posizionare gli elementi figli nella griglia con grid-column e grid-row
- Grid Container è l'elemento che ha come proprietà display: grid;
- Grid Item è l'elemento figlio del grid container
- Grid Line sono le linee che creano la struttura della griglia; possono essere sia verticali (column grid lines) che orizzontali (row grid lines)
- Grid Track spazio tra due grid lines adiacenti, si può pensare che siano le colonne e le righe della griglia



- Grid Cell spazio tra due grid lines orizzontali adiacenti e due grid lines verticali adiacenti



- Grid Area spazio totale compreso tra quattro grid lines

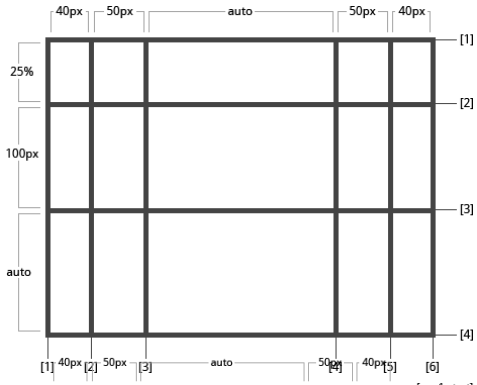


CSS Grid - Container

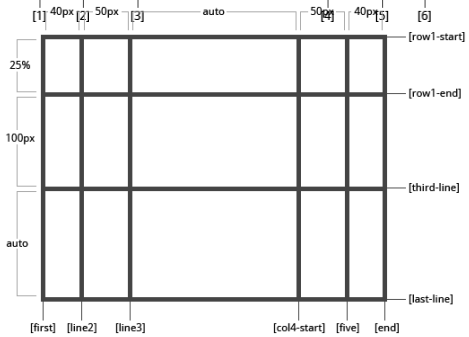
- La proprietà display definisce l'elemento container
    - grid genera una griglia block
    - inline-grid genera una griglia inline
    - subgrid nel caso in cui il grid container sia già un grid item (nel caso di griglie annidate)

- Le proprietà grid-template-columns e grid-template-rows definiscono le colonne e le righe della griglia attraverso una lista di valori:
    - <track-size> può essere una lunghezza, una percentuale o una frazione
    - <line-name> uno o più nomi a scelta (facoltativo)

```
.container {
   grid-template-columns: 40px 50px auto 50px 40px;
   grid-template-rows: 25% 100px auto;
}
```

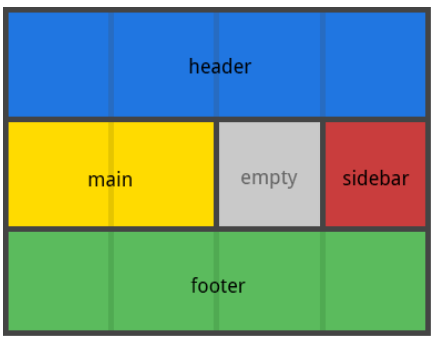



```
.container {
  grid-template-columns: [first] 40px [line2] 50px [line3] auto [col4] 50px [five] 40px [end];
  grid-template-rows: [row1-start]  25% [row1-end] 100px [third-line] auto [last-line];
}
```



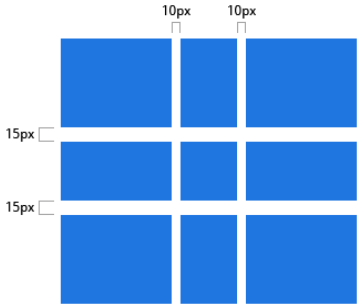
- grid-template-areas permette di definire nomi per le grid areas, specificate con grid-area
    - <grid-area-name> nome dell'area grid-area
    - . il punto sta ad indicare una cella vuota
    - none non definisce alcuna area
- Ripetere i nomi delle aree causa l'espansione del contenuto nelle celle

```
.item-a { grid-area: header; }
.item-b { grid-area: main; }
.item-c { grid-area: sidebar; }
.item-d { grid-area: footer; }
.container {
  grid-template-columns: 50px 50px 50px 50px;
  grid-template-rows: auto;
  grid-template-areas:
    "header header header header"
    "main main . sidebar"
    "footer footer footer footer";
}
```



- grid-template proprietà shorthand per impostare grid-template-rows, grid-template-columns e grid-template-areas in un'unica dichiarazione
    - none imposta tutte e tre le proprietà al rispettivo valore di default
    - subgrid imposta grid-template-row e grid-template-column su subgrid e grid-template-areas al valore di default
    - <grid-template-rows> / <grid-template-columns> impostano rispettivamente grid-template-rows e grid-template-columns ai valori definiti e grid-template-areas su none
- grid-row-gap e grid-column-gap specificano lo spessore delle linee della griglia, creando una sorta di gutter tra righe e colonne
    - <line-size> valore espresso in unità di misura
- È anche possibile utilizzare lo shorthand grid-gap passando come valori <grid-row-gap> e <grid-column-gap>

```
.container {
        grid-template-columns: 50px 50px 50px 50px;
        grid-template-rows: auto;
        grid-column-gap: 10px;
        grid-row-gap: 15px;
}
```



- justify-items allinea il contenuto all'interno di un grid-item verso destra o verso sinistra
    - start allinea il contenuto a sinistra
    - end allinea il contenuto a destra
    - center centra il contenuto
    - stretch valore default, riempie l'intera area da sinistra a destra
- justify-content allinea l'intera griglia all'interno del suo contenitore lungo l'asse delle righe; utile quando la griglia è più piccola del contenitore (es. quando vengono usate dimensioni non flessibili come i pixel)
    - start allinea la griglia a sinistra
    - end allinea la griglia a destra
    - center centra la griglia
    - stretch espande la griglia
    - space-around assegna ad ogni colonna uno spazio equo a destra e sinistra
    - space-between separa le colonne in modo che la prima stia a sinistra e l'ultima a destra
    - space-evenly distribuisce uniformemente lo spazio residuo

- align-content allinea l’intera griglia all’interno del suo contenitore lungo l’asse delle colonne
    - start allinea la griglia in alto
    - end allinea la griglia in basso
    - center centra la griglia
    - stretch espande la griglia
    - space-around assegna ad ogni riga uno spazio equo in alto e in basso
    - space-between separa le righe in modo che la prima stia a in alto e l’ultima a in basso
    - space-evenly distribuisce uniformemente lo spazio residuo

Lezione del 21/10/2020

Boostrap

Responsive design

- Tecnica di impaginazione che sfrutta le regole di disposizione e le media-query CSS3 per adattare automaticamente la visualizzazione di una pagina allo spazio effettivo presente nel browser
    - Evitando che l'utente debba scorrere la pagina orizzontalmente
- In base alla dimensione della finestra i contenuti possono essere disposti in modo diverso
    - Senza però alterare il contenuto informativo complessivo

Viewport

- La viewport è l'area di una web page visibile all'utente
- Quando questa pagina viene aperta su un dispositivo mobile, il browser cerca di simulare il comportamento che ha su un pc adattando la finestra al display
    - Questo comportamento può essere evitato inserendo il tag meta nell'header dell'hmtl con alcuni parametri

  <meta name="viewport" content="width=device-width; initial-scale=1.0; maximum-scale=1.0">
- width=device-width imposta la larghezza della pagina in base alla larghezza dello schermo, assicurando che la pagina venga visualizzata correttamente su tutti i dispositivi
- initial-scale=1.0 assicura che la pagina venga visualizzata in scala 1:1
- user-scalable=no e maximum-scale=1.0 disabilitano la possibilità di zoomare con le dita nei dispositivi mobili
    - Occorre valutare quando inserire tali vincoli

Boostrap

- Il più popolare framework CSS per la creazione di siti web
- Sistema basato su una griglia a 12 colonne
- Supporta il responsive design
- Open source
- Versione stabile corrente: 4.5.2

Mobile First

- Bootstrap permette di creare layout responsive, che adattano il loro contenuto alla dimensione degli schermi che li visualizzano
    - La filosofia di base è "mobile first": si comincia a progettare l'interfaccia perché si veda bene su un dispositivo con il display piccolo (smartphone) e poi la si estende per coprire gli altri casi

JavaScript

- Molti component Bootstrap necessitano di funzioni JavaScript, e in particolare usano jQuery e Popper.js
    - Ad esempio per gestire le animazioni degli Alert, gli stati dei Button o dei Checkbox, la visibilità della Navbar…
    - Popper.js è usato per la gestione dei Dropdown, dei Tooltip e dei Popover

Struttura Base

(se si va sul sito Boostrap trovi già tutto quello che si deve mettere dentro)

```
<!DOCTYPE html>
<html lang="en">
<head>
        <!– Questi tag devono precedere tutto il resto -->
  <meta charset="utf-8”>
  <meta name="viewport" content="width=device-width, initial-scale=1">
        <!-- Bootstrap CSS-->
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@4.5.3/dist/css/bootstrap.min.css" integrity=
"sha384-TX8t27EcRE3e/ihU7zmQxVncDAy5uIKz4rEkgIXeMed4M0jlfIDPvg6uqKI2xXr2" crossorigin="anonymous">
</head>
 <body>
  <h1>Hello, world!</h1>
  <!–- Questi file devono essere caricati al fondo della pagina e nel  seguente ordine jQuery – Popper - BootstrapJS -->
   <script src="https://code.jquery.com/jquery-3.5.1.slim.min.js"
       crossorigin="anonymous"></script>
   <script src="https://cdn.jsdelivr.net/npm/popper.js@1.16.1/dist/umd/popper.min.js"
   crossorigin="anonymous"></script>
<script src=https://cdn.jsdelivr.net/npm/bootstrap@4.5.3/dist/js/bootstrap.min.js crossorigin="anonymous"></script>
 </body>
</html>
```

Documentazione

- La documentazione di Bootstrap è suddivisa in 4 parti principali
    - Layout comprende gli approcci e le opzioni per disporre gli elementi nella pagina
    - Contenuti comprende tutti gli elementi che si possono aggiungere nella pagina (es. immagini, codice, tabelle..)
    - Componenti comprende esempi e codice di base per tutti i componenti di interfaccia predefiniti
    - Utilities comprende gli strumenti e le proprietà per modificare i singoli componenti

Struttura generale

- Bootstrap assume che lo spazio orizzontale del browser possa essere suddiviso in **12 colonne**
    - I singoli componenti vengono disposti all'interno della griglia associando loro una classe che indica il numero di colonne su cui devono estendersi
    - La larghezza delle colonne, internamente, è basata su flexbox layout
- Il sistema a griglie si applica agli elementi (div) dotati di classe "container" o "container-fluid"
    - Nel primo caso, il contenitore ha una larghezza fissa, in base al tipo di dispositivo, ed è centrato orizzontalmente
    - Nel secondo caso, il contenitore si estende al 100% della larghezza disponibile
- All'interno del contenitore devono essere presenti (direttamente) una o più righe
    - Identificate dalla classe row
- In ciascuna riga sono (direttamente) presenti elementi che si estendono su una o più colonne
    - Identificate dalla classe col-xx-y, xx può essere sm/md/lg/xl

```
<div class='container'>
        <div class='row'>
                <div class='col-xx-y'></div>
                <div class='col-xx-y'></div>
        </div>
        <div class='row'>...</div>
</div>
<div class='container'>...</div>
```

- I div all'interno di ogni riga possono essere trasformati in colonne assegnando loro la classe col-*xx*-*y*
    - xx definisce la dimensione della viewport alla quale verrà applicata la classe (sm, md, lg, xl)
    - y indica il numero di colonne su cui estendersi
- La somma dei valori di y in una riga deve essere al massimo 12
    - Se tale valore viene superato, i blocchi vengono ripartiti su più righe

Combinare le classi

- Le classi per definire le colonne possono essere combinate in un'unica classe
    - Ad esempio se vogliamo che due elementi occupino rispettivamente 4 colonne sugli schermi medium e 6 su quelli small si può scrivere

```
<div class='row'>
        <div class='col-md-4 col-sm-6'></div>
        <div class='col-md-4 col-sm-6'></div>
        <div class='col-md-4 col-sm-6'></div>
 </div>
```

Ogni classe estende il suo effetto verso le classi più in alto, quindi in questo caso la regola col-md-4 sarà valida anche nelle dimensioni lg e xl. Se non è specificato il numero di colonne, il div si estende per tutta la larghezza del contenitore.

Immagini

- Le immagini possono essere rese responsive assegnando al tag <img> la classe .img-fluid
    - Questa classe applica width:100% e height:auto permettendo all'immagine di scalare in relazione al suo elemento contenitore

CSS globale

- Bootstrap imposta di default alcuni stili
    - All'elemento <body> assegna background-color: #FFFFFF
    - Imposta su tutti gli elementi la proprietà box-sizing: border-box;
    - Sull'elemento html imposta font-family: sans-serif; e line-height: 1.15;

Colori

- All'interno del file css sono definite un insieme di classi orientate ad impostare i colori per i diversi componenti
    - Hanno la struttura *.property-usage*, dove property indica l'attributo da impostare (es. testo, sfondo, …) e usage assume uno dei seguenti valori:
        - primary, secondary, success, danger, warning, info, light, dark, muted, white

```
<div class="text-primary">
  .text-primary
</div>
<div class="text-danger">
  .text-danger
</div>
```

.text-primary

.text-danger

Spaziature

- Ci sono delle classi che permettono di definire margin e padding in maniera immediata
    - Hanno la struttura {*property}{sides}-{size}* per xs e {*property}{sides}-{breakpoint}-{size}* per sm, md, lg, xl
        - Property assume i valori di m per margin, p per padding
        - Sides assume i valori di t per top, b per bottom, l per left, r per right, x per right e left e y per top e bottom, se non specificato viene applicato sui quattro lati
        - Size assume i valori da 0 a 5 rispettivamente per 0, 0.25rem, 0.5rem, 1rem, 1.5rem, 3rem e auto per margin-auto

```
<div class="pt-4">I only have a top padding (1.5rem)</div>
<div class="p-5">I have a padding on all sides (3rem)</div>
<div class="m-5 pb-5">I have a margin on all sides (3rem) and a bottom padding (3rem)</div>
```

Componenti

- Questo gruppo comprende i componenti che non sono elementi HTML primari o che hanno un comportamento particolare in Bootstrap
    - Alert, Breadcrumb, Button, Card, Collapse…
    - Si trovano con questo url https://getbootstrap.com/docs/4.5/components/

Button

- Bootstrap fornisce molte classi predefinite per i tag di tipo <button>
    - La classe base è .btn, si può aggiungere un colore con .btn-*usage*, uno stile con bordatura con .btn-outline-*usage*, e si possono specificare delle dimensioni predefinite con .btn-sm o .btn-lg

```
<button class="btn btn-primary" type="button">Primary</button>

<button class="btn btn-outline-warning" type="button">Warning</button>

<button class="btn btn-secondary btn-lg" type="button">Large button</button>
```



Button group

- Permette di raggruppare pulsanti in un unico blocco

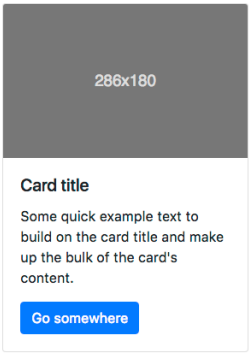
```
<div class="btn-group" role="group" aria-label="...">
 <button type="button" class="btn btn-secondary">Left</button>
 <button type="button" class="btn btn-secondary">Middle</button>
 <button type="button" class="btn btn-secondary">Right</button>
</div>
```



Card

- E' un contenitore che permette di posizionare in modo flessibile immagini e testo con un markup minimale

```
<div class="card" style="width: 18rem;">
 <img class="card-img-top" src="...">
 <div class="card-body">
  <h5 class="card-title">Card title</h5>
  <p class="card-text">Some quick example text to build on the card title and make up the bulk of the card's content.</p>
  <a href="#" class="btn btn-primary">
      Go somewhere</a>
 </div>
</div>
```



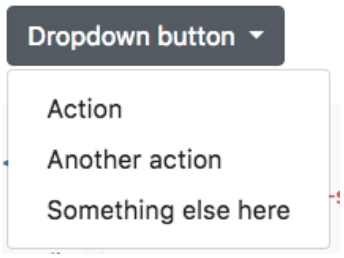
Disposizione di Card

- E' possibile raggruppare più card sulla stessa linea attraverso la classe .card-group
    - Se le card presentano un footer, questi risulteranno tutti allineati
    - La classe .card-deck ha un effetto simile ma permette di separare le card con una spaziatura
- La classe .card-columns permette di disporre le card in colonne verticali, dall'alto verso il basso e da destra verso sinistra
    - Organizza i contenuti su 3 colonne con almeno 2 card per colonna

Dropdown

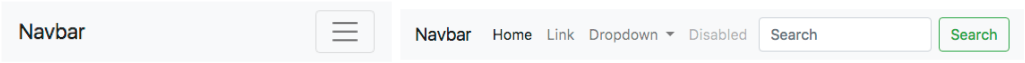
- Permette di aggiungere un menu a tendina contenente una lista di link
    - Usa la libreria Popper.js
    - Richiede l'attributo data-toggle="dropdown" sull'elemento a cui si vuole associare il menu
    - Può essere attivato con un click sull'elemento oppure tramite un frammento JavaScript

```
<div class="dropdown">
 <button class="btn btn-secondary dropdown-toggle" type="button" id="dropdownMenuButton" data-toggle="dropdown" aria-haspopup="true" aria-expanded="false">
  Dropdown button
 </button>
 <div class="dropdown-menu" aria-labelledby="dropdownMenuButton">
  <a class="dropdown-item" href="#">Action</a>
  <a class="dropdown-item" href="#">Another action</a>
  <a class="dropdown-item" href="#">Something else here</a>
 </div>
</div>
```



Navbar

- Meta-componente responsive utilizzato per la navigazione principale di un sito
    - personalizzabile tramite l'inserimento di vari elementi come il logo, i dropdown, il campo di ricerca…
    - Per i dispositivi xs è possibile usare il plugin collapse




```
<nav class="navbar navbar-expand-lg navbar-light bg-light">
 <a class="navbar-brand" href="#">Navbar</a>
 <button class="navbar-toggler" type="button" data-toggle="collapse" data- target="#navbarSupportedContent">
  <span class="navbar-toggler-icon"></span>
```

```
 </button>
 <div class="collapse navbar-collapse" id="navbarSupportedContent">
  <ul class="navbar-nav mr-auto">
   <li class="nav-item active">
    <a class="nav-link" href="#">Home</a>
   </li>
   <li class="nav-item">
    <a class="nav-link" href="#">Link</a>
   </li>
   <li class="nav-item dropdown">
    <a class="nav-link dropdown-toggle" href="#" id="navbarDropdown"
        role="button" data-toggle="dropdown>Dropdown  </a>
    <div class="dropdown-menu">…</div>
   </li>
   <li class="nav-item">
    <a class="nav-link disabled" href="#">Disabled</a>
   </li>
  </ul>
  <form class="form-inline my-2 my-lg-0">
   <input class="form-control" type="search" placeholder="Search">
   <button class="btn btn-outline-success" type="submit">Search</button>
  </form>
 </div>
</nav>
```

Nav

- Componente utilizzato per presentare contenuti alternativi nella schermata principale
  - Possono essere visualizzati come link, tab o button
  - In assenza di codice Javascript non sono interattivi




```
<ul class="nav nav-tabs">
 <li class="nav-item">
  <a class="nav-link active" href="#">Home</a>
 </li>
 <li class="nav-item">
  <a class="nav-link active" href="#">Profile</a>
 </li>
        ...
</ul>
```

Breadcrumb

- Evidenzia la posizione corrente nella mappa di navigazione del sito
    - La classe .active definisce la posizione corrente

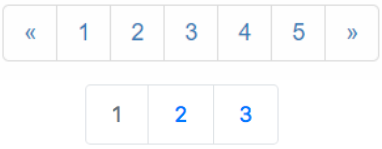
```
<ol class="breadcrumb">
 <li class="breadcrumb-item"><a href="#">Home</a></li>
 <li class="breadcrumb-item"><a href="#">Library</a></li>
 <li class="breadcrumb-item active">Library</li>
</ol>
```



Pagination

- Visualizza un menu di navigazione tra pagine
    - L'effettiva navigazione avviene solo se viene inserito dell'opportuno codice JavaScript che reagisce alla pressione dei singoli tasti e visualizza il contenuto relativo
    - Oppure associando alle singole voci link a pagine esterne
    - Tramite le classi .pagination-lg e .pagination-sm è possibile cambiare le dimensioni del menu di navigazione

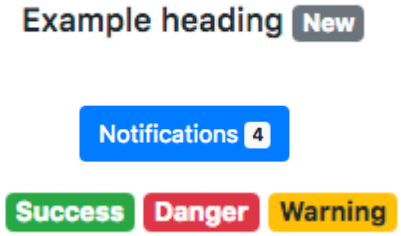
```
<nav aria-label="Page navigation example">
 <ul class="pagination">
            <!--Previous-->
  <li class="page-item">
   <a class="page-link" href="#" aria-label="Previous">
    <span aria-hidden="true">&laquo;</span>    </a>
  </li>
  <li class="page-item"><a class="page-link" href="#">1</a></li>
  <li class="page-item"><a class="page-link" href="#">2</a></li>
  <li class="page-item"><a class="page-link" href="#">3</a></li>
            ...
            <!-- Next -->
  <li class="page-item">
   <a class="page-link" href="#" aria-label="Next">
    <span aria-hidden="true">&raquo;</span>
   </a>
  </li>
 </ul>
</nav>
```



Badge

- E' un componente che consente di visualizzare una piccola porzione di testo o un numero
    - La sua dimensione viene calcolata in em relativamente al suo contenitore
    - Può essere usato come parte di un link o di un bottone come contatore

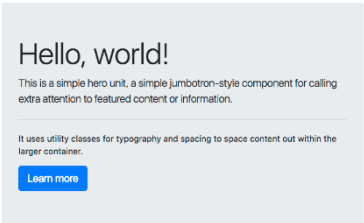
```
<h6>Example heading <span class="badge badge-secondary">New</span></h6>
<button type="button" class="btn btn-primary">
 Notifications <span class="badge badge-light">4</span>
</button>
<span class="badge badge-success">Success</span>
<span class="badge badge-danger">Danger</span>
<span class="badge badge-warning">Warning</span>
```



Jumbotron

- Crea un'area rettangolare, è utilizzato per evidenziare dei contenuti per es. contiene i messaggi di marketing
    - Per rendere il jumbotron largo quanto tutta la pagina e senza gli angoli arrotondati, è necessario aggiungere la classe .jumbotron-fluid e la classe .container o .container-fluid al suo contenuto

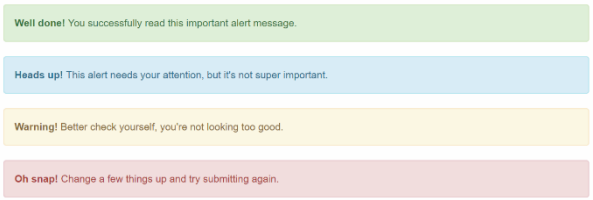
```
<div class="jumbotron">
 <h1 class="display-4">Hello, world!</h1>
 <p class="lead">This is a simple hero unit, a simple jumbotron-style component for calling extra attention to featured content or information.</p>
 <hr class="my-4">
 <p>It uses utility classes for typography and spacing to space content out within the larger container.</p>
 <a class="btn btn-primary btn-lg" href="#" role="button">Learn more</a>
</div>
```



Alert

- Crea un box di avviso a comparsa
    - Serve a comunicare messaggi di vario genere
    - Per cambiare lo stile è sufficiente aggiungere la classe .alert-{*usage*}

```
<div class="alert alert-success" role="alert">...</div>
<div class="alert alert-info" role="alert">...</div>
<div class="alert alert-warning" role="alert">...</div>
<div class="alert alert-danger" role="alert">...</div>...
```



Progress bar

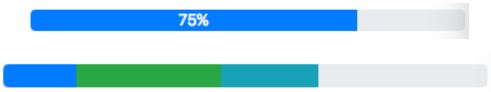
- Barra di avanzamento per fornire feedback sullo stato di un'azione
    - Si costruisce a partire da 2 elementi html e tramite il css è possibile impostare il valore di progress



```
<div class="progress">
 <div class="progress-bar" role="progressbar" aria-valuenow="60" aria-valuemin="0" aria-valuemax="100" style="width: 60%;">
 </div>
</div>
```

- E' possibile aggiungere un'etichetta alla progress
    - Si specifica il valore da visualizzare dentro il div contrassegnato con la classe .progress-bar
- Un div con la classe .progress può contenere più di un progress-bar

```
<div class="progress">
  <div class="progress-bar" role="progressbar" ...>75%</div>
</div>
<div class="progress">
  <div class="progress-bar" aria-valuenow="15"...></div>
  <div class="progress-bar bg-success" aria-valuenow="30"...></div>
  <div class="progress-bar bg-info" aria-valuenow="20"...></div>
</div>
```



Form

- I campi del form (es.input, select, textarea…) usano la classe form-control per impostare forma, dimensione e stati
- Bootstrap mette a disposizione tre differenti tipologie di layout per i form
    - Group
    - Grid
    - Inline

Group layout

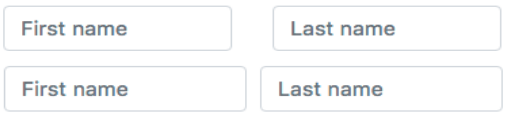
- E’ il modo più semplice di creare un form, per default imposta gli elementi verticalmente
    - Gli elementi input e label devono essere inseriti nel relativo tag ed è possibile raggrupparli in un div class="form-group"




```
<form>
 <div class="form-group">
  <label for="inputEmail">Email</label>
  <input type="email" class="form-control" id="inputEmail"          placeholder="Email">
 </div>
 <div class="form-group">
  <label for="inputPassword">Password</label>
  <input type="password" class="form-control" id="inputPassword"          placeholder="Password">
 </div>
 ...
</form>
```

Grid layout

- E’ un layout che consente di sistemare gli elementi in maniera più complessa
    - Permette di usare più colonne e assegnare diverse dimensioni a ogni elemento
    - Per sistemare gli elementi orizzontalmente basta aggiungere un elemento con la classe row o form-row (mantiene più compatti i campi di input) che a sua volta può contenere altri div con classe col




```
<form>
 <div class="row">
  <div class="col">
   <input type="text" class="form-control" placeholder="First name">
  </div>
  <div class="col">
   <input type="text" class="form-control" placeholder="Last name">
  </div>
 </div>
</form>
<form>
 <div class=”form-row">
  <div class="col">
   <input type="text" class="form-control" placeholder="First name">
  </div>
  <div class="col">
   <input type="text" class="form-control" placeholder="Last name">
  </div>
 </div>
</form>
```

- Gli elementi con classe col si distribuiscono nello spazio equamente a meno che non sia specificata una classe di tipo col-*x*
    - dove x indica il numero di colonne da occupare all'interno della riga
    - Usando la classe col-auto l'elemento occupa lo spazio "di cui ha bisogno", cioè la sua dimensione è definita in base al suo contenuto

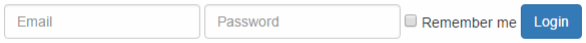
```
<form>
 <div class="form-row">
  <div class="col-7">
   <input type="text" class="form-control" placeholder="City">
  </div>
  <div class="col">
   <input type="text" class="form-control" placeholder="State">
  </div>
  <div class="col">
   <input type="text" class="form-control" placeholder="Zip">
  </div>
 </div>
</form>
```



Inline layout

- Per creare un form inline è necessario inserire il tag <form> e assegnargli la classe form-inline
    - Gli elementi vengono disposti inline solo sui dispositivi che hanno larghezza maggiore di 576px

```
<form class="form-inline">
  <div class="form-group">
    <label class="sr-only" for="inputEmail">Email</label>
    <input type="email" class="form-control" id="inputEmail"       placeholder="Email">
  </div>
  <div class="form-group">
    <label class="sr-only" for="inputPassword">Password</label>
    <input type="password" class="form-control" id="inputPassword"         placeholder="Password">
  </div>
  <div class="checkbox">
    <label><input type="checkbox"> Remember me</label>
  </div>
  <button type="submit" class="btn btn-primary">Login</button>
</form>
```



Testo di suggerimento

- Per aiutare l'utente a capire il significato del campo di input o le regole adottate si può usare un elemento con la classe form-text

```
<form class="form-inline">
 <label for="inputPassword">Password</label>
 <input type="password" id="inputPassword" class="form-control">
 <small id="passwordHelpBlock" class="form-text text-muted">
   Must be 8-20 characters long.
 </small>
</form>
```

Checkbox e radio button

- All'interno di un form è possibile inserire checkbox e radio button
  - uno sotto l'altro, verticalmente
  - inline
- Per posizionare le checkbox e i radio button in maniera verticale è sufficiente inserire gli elementi all'interno di un e applicare la classe form-check sul contenitore

```
<div class="form-check">
  <input class="form-check-input" type="checkbox" value="" id="cb1">
  <label class="form-check-label" for="cb1">Default checkbox </label>
</div>
<div class="form-check">
  <input class="form-check-input" type="checkbox" value="" id="cb2"          disabled>
<label class="form-check-label" for="cb2">Disabled checkbox</label>
</div>
```



- Per posizionare le checkbox e i radio button in inline è necessario aggiungere la classe .form-check-inline ad ogni elemento marcato con la classe .form-check




```
<!-- Inline checkbox -->
<div class="form-check form-check-inline">
  <input class="form-check-input" type="checkbox"    id="icb1" value="option1">
  <label class="form-check-label" for="icb1">1</label>
</div>
<div class="form-check form-check-inline">
  <input class="form-check-input" type="checkbox" id="icb2"    value="option2">
  <label class="form-check-label" for="icb2">2</label>
</div>
<div class="form-check form-check-inline">
  <input class="form-check-input" type="checkbox" id="icb3"    value="option3» disabled>
  <label class="form-check-label" for="icb3">3</label>
</div>
<!– Per i checkbox il codice è lo stesso ad eccezione dell'attributo type che assume il valore radio-->
```

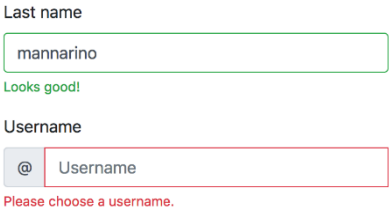
Input disabilitati

- Per inserire input di testo disabilitati basta aggiungere l'attributo booleano disabled


```
<form>
   <input type="text" class="form-control" placeholder="Disabled input" disabled>
</form>
```

Stati di validazione

- Bootstrap fornisce un meccanismo potente per progettare e presentare i diversi stati di convalida
- La validazione è applicata tramite le pseudo-classi CSS :invalid e :valid
  - Sugli elementi di tipo <input>, <select> e <textarea>
- Per personalizzare la validazione occorre aggiungere al form l'attributo booleano novalidate
  - In questo modo si disabilita il comportamento di default del browser



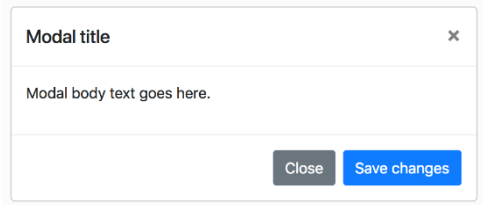
```
<form class="needs-validation" novalidate>
 <div class="form-row">
  <div class="col-md-4 mb-3">
   <label for="validationCustom02">
    Last name
   </label>
   <input type="text" class="form-control" id="validationCustom02"          placeholder="Last name" value="Otto" required>
   <div class="valid-feedback"> Looks good! </div>
  </div>
  <div class="col-md-4 mb-3">
   <label for="validationCustomUsername">Username</label>
   <div class="input-group">
    <div class="input-group-prepend"><span class="input-group-text"          id="inputGroupPrepend">@</span>
    </div>
     <input type="text" class="form-control"    id="validationCustomUsername" placeholder="Username" required>
     <div class="invalid-feedback"> Please choose a username. </div>
    </div>
 </div>
</form>
```

Plugin JavaScript

- Bootstrap mette a disposizione del programmatore una serie di plugin JavaScript che automatizzano alcuni comportamenti
    - Tali plugin richiedono la presenza della libreria JQuery che deve essere caricata prima del corrispondente file bootstrap

Modal

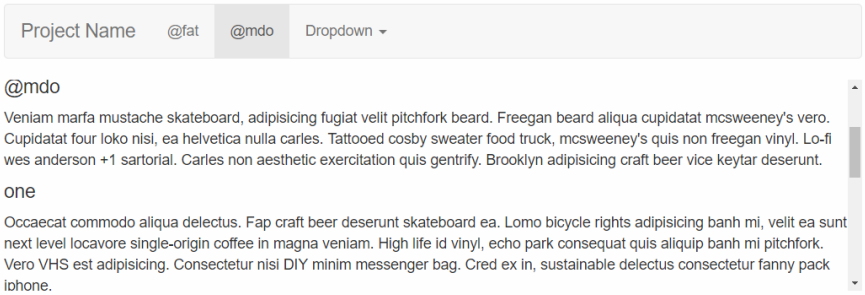
- Fa comparire una finestra di dialogo sopra tutto il resto
    - È possibile visualizzarne solo una alla volta
    - Richiede un frammento di codice JavaScript per dare il focus ad un elemento contenuto al suo interno




```
<!-- Button trigger modal -->
<button type="button" class="btn btn-primary" data-toggle="modal" data-target="#exampleModal"> Launch demo modal
</button>
<!-- Modal -->
<div class="modal" tabindex="-1" role="dialog">
 <div class="modal-dialog" role="document">
  <div class="modal-content">
   <div class="modal-header">
    <h5 class="modal-title">Modal title</h5>
    <button type="button" class="close" data-dismiss="modal" aria-     label="Close"> <span aria-
hidden="true">&times;</span> </button>
   </div>
   <div class="modal-body">
    <p>Modal body text goes here.</p>
   </div>
   <div class="modal-footer">
    <button type="button" class="btn btn-secondary" data-    dismiss="modal">Close</button>
    <button type="button" class="btn btn-primary">Save       changes</button>
   </div>
  </div>
 </div>
<div>
```

Scrollspy

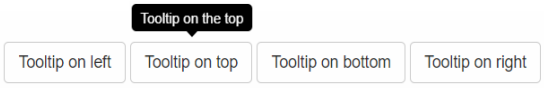
- Serve ad aggiornare automaticamente la voce di una barra di navigazione in base al contenuto attualmente visualizzato nella pagina e quindi alla posizione di scorrimento



Tooltip

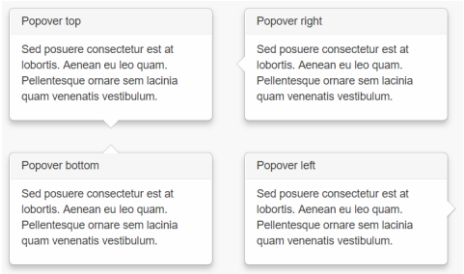
- Consente di visualizzare dei fumetti con delle informazioni, detti tooltips, al passaggio del mouse su un elemento
    - Se i tooltip sono vuoti non vengono visualizzati

```
<button type="button" class="btn" data-toggle="tooltip" data-placement="top" title="Tooltip on top">Tooltip on top</button>
$(function() {
  $('.btn').tooltip();
});
```



Popover

- Aggiunge un fumetto con contenuti secondari a qualsiasi elemento
    - Simile al tooltip ma consente di inserire layout più elaborati

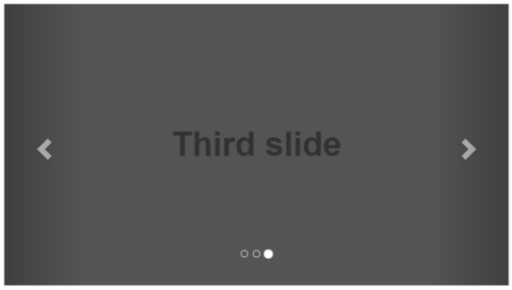


Collapse

- Permette di utilizzare una serie di classi per gestire i comportamenti on/off (toggle)
    - Dando a un elemento l'attributo data-toggle="collapse" e assegnando un href corrispondente all'elemento da visualizzare, oppure utilizzando data-target
    - Le classi applicate al target sono 3
        - .collapse nasconde l’elemento
        - .collapsing è applicato durante la transizione
        - .collapse.show mostra l’elemento

Carousel

- Crea uno slideshow di più elementi e li visualizza uno dopo l'altro



Lezione del 27/10/2020

Scalable Vector Graphics

SVG – Scalable Vector Graphics

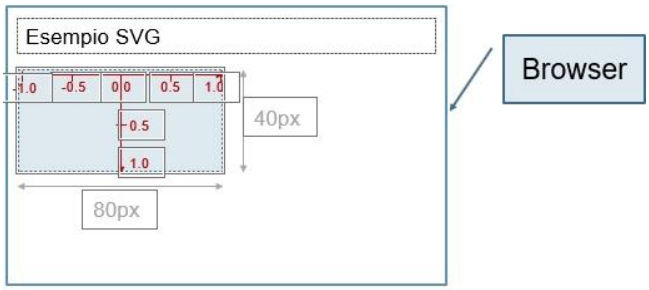
- Formato di grafica vettoriale che permette di descrivere contenuti visuali in modo analogo a quanto fa HTML per i contenuti testuali
    - Si basa sull'uso di tag annidati, ciascuno dei quali può essere decorato mediante opportuni attributi
- Il formato SVG può essere incorporato all'interno di un documento HTML
    - Sia sotto forma di immagine, tramite elementi <img> il cui attributo src punti ad una risorsa di tipo svg
    - Sia direttamente nel documento, inserendo elementi di tipo <svg>: in questo caso è possibile utilizzare regole CSS per specificare la presentazione del contenuto

L'elemento <svg>

- Radice della gerarchia della componente grafica
    - Definisce un sistema di riferimento per i contenuti grafici attraverso l'attributo **viewBox**
    - Definisce uno spazio, nelle coordinate del browser in cui tale viewBox viene mappata attraverso gli attributi **x**, **y**, **width**, **height**
    - Definisce una modalità per associare le coordinate della viewBox alle coordinate del browser tramite l'attributo **preserveAspectRatio**

Sistemi di riferimento

```
<html>
 <head>…</head>
 <body>
  <h1>Esempio SVG</h1>
  <svg width="80" height="40"" viewBox="-1 0 2 1">…</svg>
 </body>
</html>
```



N.B. x aumenta da sinistra a destra, sulla y dall'alto verso il basso
*N.B.2 gli elementi che nonhanno contenuto dentro a svg si scrivono: <line … **/>***

Contenuto

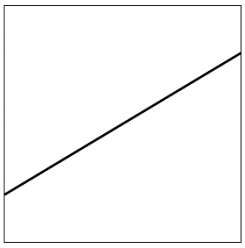
- All'interno dell'elemento <svg> possono essere inseriti
    - Elementi di tipo forma (<circle>, <line>, <text>, …)
    - Elementi strutturali (<defs>, <g>, …)
    - Elementi descrittivi (<desc>, <metadata>, <title>)
    - Definizioni di gradienti (<linearGradient>, <radialGradient>, …)
    - Definizioni di animazioni (<animate>, <animateColor>, <animateMotion>, …)
    - Definizione di filtri, maschere, contorni da ritagliare, font, ancore, pattern, stili, …

- Altri elementi HTML (<foreignObject>)

Segmenti

- L'elemento <line> permette di descrivere un segmento
  - Le sue estremità sono definite dagli attributi x1, y1, x2, y2
  - L'attributo stroke ne definisce il colore, stroke-width lo spessore, stroke-opacity l'opacità (tra 0 e 1)
  - La forma delle sue estremità è definita dall'attributo stroke-linecap, il tratteggio dagli attributi stroke-dasharray e stroke-dashoffset

```
<svg
 viewBox="0 0 100 100" >
 <line
   x1="0" y1="80"
   x2="100" y2="20"
   stroke="black"
 />
</svg>
```



Poligonali

- L'elemento <polyline> descrive una linea poligonale
  - Composta da una sequenza di vertici contenuti nell'attributo points, il cui valore contiene una sequenza di coppie di valori, separate tra loro da spazi
  - I valori presenti nelle singole coppie sono separati da virgole
  - Gli attributi di tipo stroke-… controllano la presentazione del perimetro
  - Se il punto iniziale è diverso da quello finale, il perimetro è aperto, ma l'interno della poligonale potrebbe apparire colorato se l'attributo fill non è posto a none

```
<svg
 viewBox="0 0 200 100" >
<polyline
  points="0,100 50,25 50,75 100,0"
  stroke="orange" />
<polyline
  points="100,100
150,25 150,75 200,0"
  fill="none"
  stroke="orange"  7>
</svg>
```



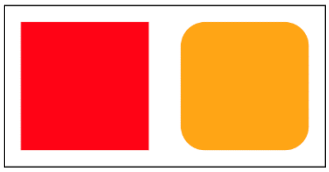
- L'elemento <polygon> definisce una forma chiusa, in cui – automaticamente – il punto iniziale è raccordato al punto finale
  - Gli attributi sono gli stessi delle poligonali

Rettangoli

- L'elemento <rect> definisce un rettangolo parallelo agli assi di riferimento
  - Gli attributi x, y, width e height ne definiscono la geometria

- Gli attributi rx e ry indicano il raggio di curvatura da usare per smussarne gli angoli
- Gli attributi di tipo stroke-… controllano la presentazione del perimetro
- Gli attributi fill, fill-opacity, fill-rule ne controllano il riempimento

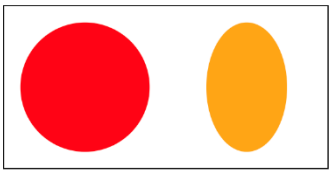
```
<svg
 viewBox="0 0 200 100" >
 <rect x="10" y="10"
  width="80"
  height="80"
  fill="red" />
 <rect x="110" y="10"
   width="80"
   height="80"
   rx="15"
   fill="orange" />
</svg>
```



Cerchi e ellissi

- Gli elementi <circle> e <ellipse> permettono di visualizzare le corrispettive figure geometriche
  - In entrambi i casi, gli attributi cx e cy determinano l'incrocio degli assi
  - Nel caso del cerchio, il raggio viene specificato tramite l'attributo r, mentre per l'ellisse sono indicati le lunghezze dei due semiassi, rx e ry
  - Gli attributi di presentazione restano gli stessi delle figure precedenti

```
<svg
 viewBox="0 0 200 100" >
 <circle cx="50"
  cy="50"
  r="40"
  fill="red" />
 <ellipse
   cx="150" cy="50"
   rx="25" ry="40"
   fill="orange" />
</svg>
```

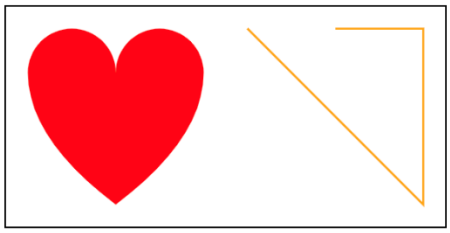


Cammini

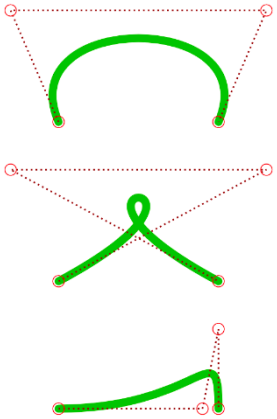
- L'elemento <path> costituisce il modo più generico di descrivere una forma
  - L'attributo d contiene una sequenza di comandi costituiti da una lettera seguita da eventuali parametri che ne specifica il contorno
- I comandi si dividono in 6 categorie
  - Spostamento del cursore logico (M *x,y* | m *dx,dy*)
  - Tracciamento di linee (L *x,y* | l dx,dy | H *x* | h *dx* | V *y* | v *dy*)
  - Curve quadratiche (Q x_1,y_1 *x,y* | q dx_1,dy_1 *dx,dy* | T *x,y* | t *dx,dy*)
  - Curve cubiche (C x_1,y_1 x_2,y_2 *x,y* | c dx_1,dy_1 dx_2,dy_2 *dx,dy* | S x_1,y_1 *x,y* | s dx_1,dy_1 *dx,dy*)
  - Archi ellittici (A *rx ry a* f_1 f_2 *x,y* | a *rx ry a* f_1 f_2 *dx,dy*)

- Chiusura del cammino (Z | z)

```
<svg
 viewBox="0 0 200 100" >
 <path fill="red"
   d={`M 10,30
   A 20,20 0,0,1 50,30
   A 20,20 0,0,1 90,30
   Q 90,60 50,90
   Q 10,60 10,30 z`}/>
 <path fill="none"
   stroke="orange"
   d={`M 110,10
     l 80,80
     v -80
     h -40`} />
</svg>
```

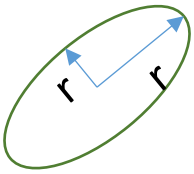


Curve di Bezier



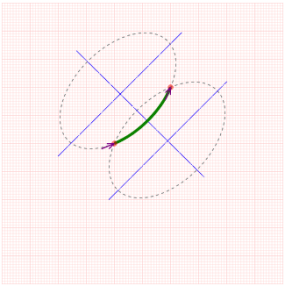
- Famiglia di curve parametriche "dolci" che permettono la semplice modellazione di curve, controllando
  - Punto iniziale e finale
  - Tangente al punto iniziale e finale
- Vengono espresse tramite punti di controllo
  - Un solo punto di controllo per le curve quadratiche
  - Due punti per le curve cubiche

Archi ellittici



- Formula generale: A r_x r_y a f_1 f_2 x y
- Si costruisce un'ellisse con semiassi r_x e r_y
- La si ruota di a gradi rispetto all'asse x
- Si impone che passi per il punto corrente e per il punto dato (x, y)

```
<path d="M40 50 A 25 15 -45 0 0 60 30" />
```



- Si può ottenere
  - Nessuna soluzione
  - 4 soluzioni
- Nel primo caso
  - I raggi vengono dilatati fino a formare l'ellisse più piccolo passante per i due punti
- Nel secondo caso
  - Si usano i parametri f_1 e f_2 per scegliere l'arco

Testo

- L'elemento <text> permette di visualizzare le forme relative ad un contenuto testuale
    - Queste possono essere ritagliate, colorate, deformate, …, come se si trattasse di una geometria qualsiasi
- Il testo visualizzato è quello contenuto all'interno dell'elemento stesso
    - Oltre agli attributi di presentazione tipici di tutte le forme con contenuto, è possibile modificare il punto di ancoraggio del testo, tramite l'attributo text-anchor

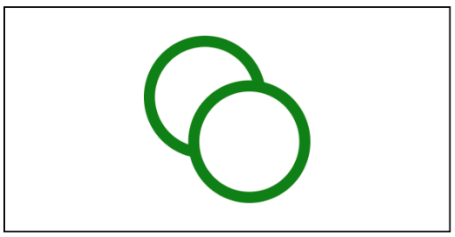
```
<svg
 viewBox="0 0 240 180" >
 <style>
.small {font: italic
      13px sans-serif;}
.heavy {font: bold 30px
       sans-serif;}
.r {font: italic 40px
       serif; fill: red;}
</style>
 <text x="20" y="35"
  class="small">My</text>
 <text x="40" y="35"
 class="heavy">cat</text>
 <text x="55" y="55"
 class="small">is</text>
 <text x="65" y="55"
 class="r">Grumpy!</text>
</svg>
```



Gruppi

- L'elemento <g> permette di raggruppare più forme tra loro
    - Gli elementi di presentazione applicati al gruppo vengono automaticamente ereditati dai singoli elementi contenuti al suo interno

```
<svg
 viewBox="0 0 200 100">
 <g fill="white"
   stroke="green"
   stroke-width="5">
  <circle cx="90"
    cy="40" r="25" />
  <circle cx="110"
    cy="60" r="25" />
 </g>
</svg>
```

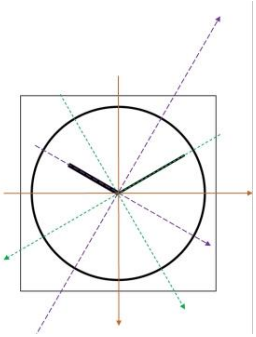


Trasformazioni geometriche

- Tutti gli elementi di tipo forma, così come i gruppi, permettono di specificare l'attributo transform
    - Questo indica che le coordinate dell'elemento cui è applicata la trasformazione sono espresse in un sistema di riferimento differente rispetto a quello originale

- Possibili valori dell'attributo
    - translate(dx, dy) – indica una traslazione
    - scale(sx, sy) – indica un ingrandimento/riduzione di scala
    - rotate(a, cx, cy) – indica una rotazione di a gradi attorno al punto di coordinate cx, cy
    - skewX(a) – indica una rotazione di a gradi dell'asse Y (l'asse X resta fermo)
    - skewY(a) – indica una rotazione di a gradi dell'asse X (l'asse Y resta fermo)
- Più operazioni possono essere applicate in cascata, separandole da spazi
    - Attenzione all'ordine!

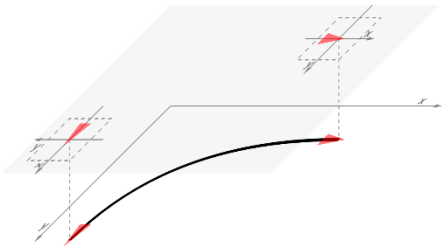
```
<svg
 viewBox="-10 -10 20 20">
 <g stroke="black"
  stroke-width="0.2">
  <circle r="9"
    fill="white"/>
  <line x2="0" y2="-8"
   transform="rotate(60)" />
  <line x2="0" y2="-8"
    transform="rotate(300)
        scale(2 0.75)" />
 </g>
</svg>
```



Marcatori

- E' possibile associare ai punti iniziali, finali e intermedi di un cammino opportuni marcatori
    - Definiti come oggetti SVG entro-contenuti nella sezione <defs></defs>
- Ogni elemento dotato di un bordo può indicare quali marcatori intende utilizzare
    - Con gli attributi marker-start, marker-end, marker-mid
- Un marcatore è introdotto dall'elemento <marker>
    - Questo deve avere un id (per poterlo referenziare), una viewbox (per definire un sistema di riferimento locale), una dimensione (markerWidth / markerHeight) a cui sarà scalato ed un orientamento
- Il marcatore viene sovrapposto al tratto di un elemento grafico in corrispondenza dei punti estremi ed intermedi
    - Facendo coincidere le coordinate refX e refY del sistema di riferimento del marcatore con il punto su cui deve essere applicato (default 0, 0)
    - Ruotando il marcatore in modo da allineare il suo asse X con la tangente al segmento cui viene applicato (eventuale ribaltamento punto iniziale)
    - Scalando il marcatore in base alle dimensioni indicate

```
<svg viewBox="0 0 100 100">
<defs>
<marker id="m" viewBox="-50 -50 100 100"
makerWidth="16" markerHeight="16"
orient="auto-start-reverse">
<path d="M -30 -10 L 10 0 L 30 -10" />
</marker>
</defs>
<path d = "M 60 10 Q 10 10 10 40"
marker-start="url(#m)"
marker-end="url(#m)" />
</svg>
```

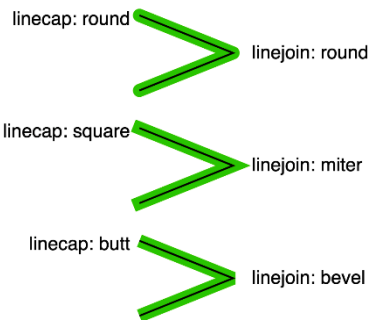


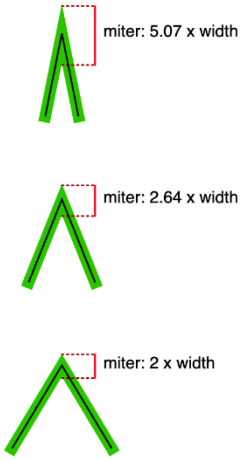
SVG e CSS

- Gli elementi di stile possono anche essere applicati direttamente ai singoli elementi SVG
    - <circle r="10" **stroke="blue"** />
- In caso di collisione con quanto definito a livello di CSS, **la regola CSS prevale** sui valori assegnati agli attributi dell'elemento
    - E' opportuno utilizzare le classi CSS anche per contenere la quantità di informazioni nei singoli elementi

Stilizzare elementi SVG

- Proprietà degli elementi geometrici
    - fill,
      fill-opacity, fill-rule,
    - marker,
      marker-start, marker-mid, marker-end,
    - stroke,
      stroke-width, stroke-dasharray, stroke-dashoffset,
      stroke-linecap, stroke-linejoin, stroke-miterlimit,
      stroke-opacity
    - text-rendering





FONTI UTILI PER AVERE GiA' CODICI SVG: https://thenounproject.com/term/svg-file/138223/

- Proprietà del testo
    - text-anchor,
      alignment-baseline, baseline-shift, dominant-baseline
    - glyph-orientation-horizontal,
      glyph-orientation-vertical
    - kerning

LA LEZIONE r12 è utile per capire meglio tutta quest'ultima parte!